Predictive Model Plan: Geldium Delinquency Risk

Model logic drafted with GenAI support and checked against my EDA of the Geldium dataset (500 customers, 19 columns).

1. Model Logic (Generated with GenAI)

Goal. Predict whether a customer will have a delinquent account (Delinquent_Account = 1) so Geldium can prioritise early outreach. This is a binary classification problem.

Chosen model (summary)

I would use a regularised logistic regression as the primary model. It turns each customer's financial and payment-history variables into a probability of delinquency between 0 and 1, and every coefficient can be read and explained. A random forest or gradient-boosted model would be built alongside it as a challenger, not as the live model.

Top 5 input features

Feature	Why it should matter	What my EDA showed
Credit_Utilization	High use of available credit signals financial stress	Mean 49%; 4 customers above 100% (data-quality flag)
Debt_to_Income_Ratio	Measures repayment burden against income	Mean about 0.30, range 0.10 to 0.55
Credit_Score	Summary of past credit behaviour	Range 301 to 847; 2 missing values
Payment history (Month_1 to Month_6)	Recent lateness is a leading indicator; summarised as a late-payment count plus the latest months	Roughly one third each of On-time, Late, Missed
Missed_Payments	Direct count of misses in the last 12 months	Values 0 to 6, no missing data

Note: these five are chosen on credit-risk reasoning. In this sample, no single feature has a correlation with delinquency above 0.06 (see Section 3), so the ranking should be re-tested on a larger dataset.

Workflow: from raw data to risk prediction

- **Ingest:** load the customer table; Delinquent_Account is the target (16% delinquent, 84% not).
- **Clean:** standardise Employment_Status labels (EMP, Employed and employed all mean the same thing); flag utilization above 100%; impute missing Income (39), Loan_Balance (29) and Credit_Score (2) with the median and add a "was missing" flag.
- **Engineer:** convert Month_1 to Month_6 to 0/1/2, then create late_count (months late or missed) and last_3_months_late; one-hot encode categorical columns; scale numeric columns.
- **Split:** 80/20 stratified train/test split so both sets keep the 16% delinquency rate; use 5-fold cross-validation on the training set.
- **Train:** logistic regression with L2 regularisation and class weights to handle the imbalance.
- **Predict:** output a delinquency probability, then map it to a risk band (Low / Medium / High) using a threshold chosen for the business's tolerance for missed risky customers.
- **Explain and monitor:** show the top drivers of each customer's score; track performance and drift monthly.

Pseudocode

```
load data; target = Delinquent_Account
clean Employment_Status labels; flag Credit_Utilization > 100%
map Month_1..Month_6: On-time=0, Late=1, Missed=2
```

```
late_count = number of months with value >= 1
impute numeric NaN with median; one-hot encode categoricals; scale numerics
X_train, X_test = stratified_split(data, test=0.2)
model = LogisticRegression(penalty="l2", class_weight="balanced")
cross_validate(model, X_train, scoring=[AUC, recall, precision])
model.fit(X_train); p = model.predict_proba(X_test)[:,1]
risk_band = High if p >= t_high; Medium if p >= t_med; else Low
report AUC, recall, precision, F1 + fairness metrics by group
```

2. Justification for Model Choice

Logistic regression is the right starting model for Geldium because it balances accuracy with transparency, which matters most in lending. Each feature has a coefficient that shows whether it raises or lowers delinquency risk, so analysts can explain any score to a customer, an auditor or a regulator, supporting adverse-action and fair-lending requirements. It handles numeric financial variables well, trains in seconds, is simple to deploy and monitor, and produces calibrated probabilities that the business can turn into risk bands and collection strategies. The trade-off is that it assumes mostly linear relationships and can miss complex interactions, so a random forest or gradient-boosted model would run as a challenger, and would replace the logistic model only if it were clearly better and explainable (for example via SHAP values). With only 500 records, a simple model is also less likely to overfit.

Criterion	Logistic regression	Decision tree / random forest	Gradient boosting / neural net
Accuracy	Good baseline	Better on non-linear patterns	Often highest
Transparency	High	Tree: high; forest: low	Low
Ease of deployment	Very easy	Moderate	Harder
Regulatory fit	Strong	Moderate	Needs extra explainability
Data needs	Works with small data	Needs more data	Needs the most data

3. Evaluation Strategy

Baseline reality check from my data

Before planning metrics, I ran a quick 5-fold cross-validation on the dataset. Cross-validated AUC was about 0.45 for logistic regression, 0.41 for random forest and 0.40 for gradient boosting; an AUC of 0.50 is a coin flip. All feature correlations with delinquency were below 0.06, and even Missed_Payments was not positively related to it. So, on this sample, none of the models predicts delinquency better than chance, and the more complex models did no better than the simple one. Possible causes are a small sample (only about 80 delinquent customers), a target that depends on information not in the table, or data that is partly synthetic. This is a key finding: the plan below defines how the model must be judged before it is trusted, and the model should not be deployed on this data.

Accuracy and reliability metrics

Metric	How to interpret it	Target (to agree with Geldium)
Accuracy	Share of correct predictions. Misleading here: predicting "no one is delinquent" already scores 84%.	Reported, never used alone
Recall	Share of truly delinquent customers caught. Most important, since a missed risky customer costs money.	About 60% or higher at the chosen threshold

Metric	How to interpret it	Target (to agree with Geldium)
Precision	Share of flagged customers who really are delinquent. Controls wasted outreach cost.	Set by collections capacity
F1 score	Balances precision and recall in one number.	Compare against the baseline
AUC-ROC	How well the model ranks risky above safe customers (0.5 = chance, 1.0 = perfect).	At least 0.70 before deployment
Confusion matrix	Shows exact counts of false positives and false negatives.	Review at each threshold

Reliability checks: stratified cross-validation, a held-out test set, bootstrapped confidence intervals around AUC (essential with a small sample), a calibration plot, and monthly drift monitoring after launch.

When the model needs improvement

- AUC below 0.70, or a confidence interval that includes 0.50.
- Recall too low at an acceptable precision.
- Large gaps in performance between customer groups.
- Performance falling on new data (drift). Responses: collect more data, engineer better features (e.g. payment trends, utilization change), or try the challenger models.

Bias and fairness checks

Groups to test: Age bands, Location, Employment_Status (and Credit_Card_Type). Location and Employment_Status can act as proxies for protected characteristics, so they get extra scrutiny.

Fairness check	What it measures
Approval / flag rate by group	Demographic parity difference between groups
Disparate impact ratio	Flag-rate ratio between groups; flag if below 0.80 (four-fifths rule)
True-positive-rate gap	Equal opportunity: are risky customers caught equally across groups?
False-positive-rate gap	Are some groups wrongly flagged more often?
Calibration by group	Does a 30% score mean about 30% for every group?

Mitigation if gaps appear: remove or re-test proxy features, reweigh training samples, tune thresholds per group where legally allowed, and re-check after each change. Group-level results in this dataset are small (roughly 75 to 120 customers per group), so differences must be tested for statistical significance before drawing conclusions.

Ethical considerations

- Scores support human decisions (early outreach, repayment plans); they should not trigger automatic denials or penalties.
- Customers affected by a decision should receive a plain-language reason based on the top drivers.
- Collect and use only data needed for the purpose, and protect customer privacy.
- Keep documentation and a regular review cycle so the model is auditable.
- Be open about uncertainty: a model that performs near chance must not be presented as reliable.

How GenAI was used

GenAI helped draft the model options, pipeline outline, pseudocode and the fairness-metric list. I then checked the suggestions against the dataset and revised them: for example, the initial feature ranking was adjusted after EDA showed weak signal, and the data-quality fixes (inconsistent employment labels, missing values, utilization above 100%) came from inspecting the data directly.